



Introduction to Algorithms

By Prince Lesapo



What Is an Algorithm?

An algorithm is just a set of steps for making a decision. That's it. Nothing magical about it.

The human version

You have 1000 student records. Study hours, attendance, past grades. Someone says: tell me who's going to pass or fail.

You'd sort. You'd look. You'd make rules in your head. If less than 2 hours study AND missed most classes... probably failing.

The algorithm version

Does exactly the same thing. Learns those rules from the data. Then applies them to every new student it sees.

Faster. More consistent. Works on a million students, not just five.

The algorithm didn't replace the thinking. It automated it.

Three Families of Algorithms

Supervised Learning

Your friend has the memorandum. Every answer you give, they correct you. You learn from those corrections.

You give the algorithm data where you already know the answer. It learns from that. Then predicts on new data it's never seen.

Examples:

- Decision Trees
- Random Forests
- Linear Regression

Unsupervised Learning

No memorandum. No answers. Just a pile of past exam papers.

You start spotting patterns yourself. This question always comes up. These topics always appear together. Nobody told you that.

Examples:

- K-Means Clustering
- PCA

Reinforcement Learning

Learning to ride a bike. No textbook. You just try, fall, adjust, try again.

Every time you stay up longer, something tells you you're getting it. Every time you fall, you know that didn't work.

Examples:

- YouTube recommendations
- Game-playing AI

Today we go deep on Supervised Learning. Specifically: Decision Trees.



A Familiar Example

Imagine DIRISA had been collecting data on every student who ever came through this training.

Did they attend every session?

Feature

Did they do the practicals?

Feature

Did they ask questions?

Feature

Did they find extra resources outside the training?

Feature

Did they actually engage, or just sit there?

Feature

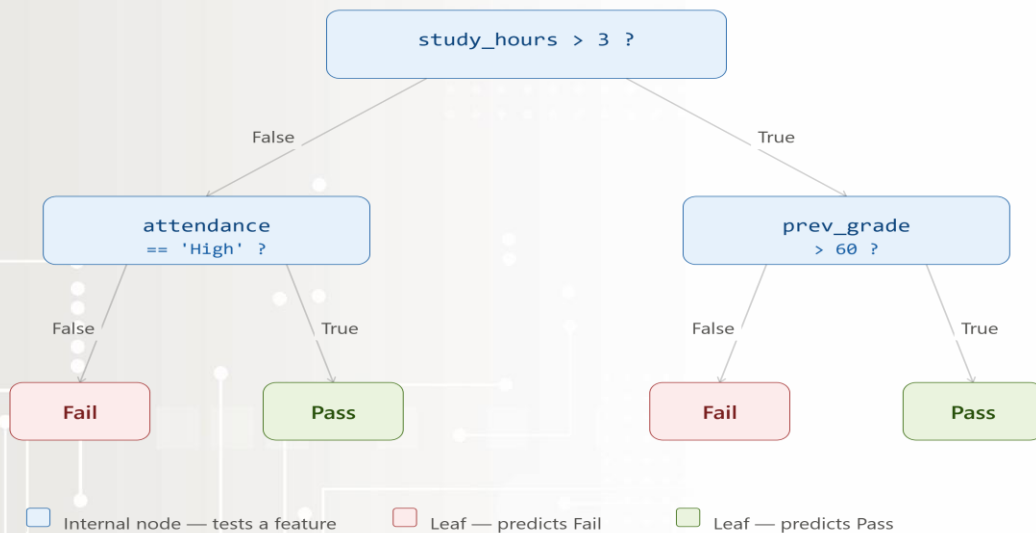
Final column: Did they make it to nationals? ← This is the target variable



What Is a Decision Tree?

"A Decision Tree is a non-parametric supervised learning algorithm with a hierarchical tree structure consisting of a root node, branches, internal nodes and leaf nodes, employing a divide-and-conquer strategy through recursive partitioning to classify or predict outcomes."

It is essentially An algorithm that makes decisions by asking questions. One at a time. Until it reaches a final answer.



Reading the tree:

Internal node

`study_hours > 3 ?` ← tests a feature

Branch

`True / False` ← outcome of the test

Leaf node: Pass

`class label` ← final prediction

Leaf node: Fail

`class label` ← final prediction

How did it know to ask that question first? It tried every possible question and measured which split reduced the most confusion in the data. That measurement is called Information Gain — and it starts with Entropy. That is what is next.

Meet the Dataset

Two features. One target variable. Five possible questions.

Student	Study hrs	Attendance	Result
S1	5 hrs	High	PASS
S2	1 hr	Low	FAIL
S3	4 hrs	High	PASS
S4	2 hrs	Low	FAIL
S5	3 hrs	High	PASS
S6	1 hr	High	FAIL
S7	5 hrs	Low	PASS
S8	2 hrs	High	FAIL
S9	4 hrs	Low	PASS
S10	1 hr	Low	FAIL

Target variable: Result (Pass / Fail): This is what we are predicting.

Features available to the tree:

Study Hours (numerical)

Unique values: 1, 2, 3, 4, 5 hrs

Possible questions:

`study_hours > 1 ?`

`study_hours > 2 ?`

`study_hours > 3 ?`

`study_hours > 4 ?`

Attendance (categorical)

Values: High, Low

Possible question: `attendance == 'High' ?`

5 possible questions total. The tree evaluates all 5, calculates information gain for each, and picks the winner.

How Does the Tree Know What to Ask First?

The algorithm needs a way to figure out which question is most useful.

Entropy: $H(S) = -\sum p_i \log_2 p_i$

Measures how mixed up a group is.

- Pure group = low entropy.
- Mixed group = high entropy.

All 10 students pass $\rightarrow H(S) = 0$.

5 pass, 5 fail $\rightarrow H(S) = 1.0$ (maximum)

Information Gain: $IG = H(\text{parent}) - \text{weighted } H(\text{children})$

How much does asking this question reduce the confusion?

Higher gain = better question to ask first

The algorithm tries every possible question on every feature and picks the one with the highest information gain. That becomes the first split.

Pure group



All same $\rightarrow$ Entropy = 0

Mixed group



Half and half $\rightarrow$ Entropy = 1.0

Which question should the tree ask first?

We have **5** possible questions. Let's calculate one to see how information gain actually works. We'll try **study_hours > 3**.

Student	Study hrs	Attendance	Result
S1	5 hrs	High	PASS
S2	1 hr	Low	FAIL
S3	4 hrs	High	PASS
S4	2 hrs	Low	FAIL
S5	3 hrs	High	PASS
S6	1 hr	High	FAIL
S7	5 hrs	Low	PASS
S8	2 hrs	High	FAIL
S9	4 hrs	Low	PASS
S10	1 hr	Low	FAIL

Step 1: Entropy of the whole group

5 pass, 5 fail out of 10 students → $H = 1.0$ (maximum uncertainty)

$$H = -(0.5 \times \log_2 0.5) - (0.5 \times \log_2 0.5)$$

$$p(\text{pass}) = 5/10 = 0.5$$

$$p(\text{fail}) = 5/10 = 0.5$$

$$H = -(0.5 \times -1.0) - (0.5 \times -1.0) = 1.0$$

Step 2: Split on study hours > 3

YES (S1,S3,S7,S9): 4 pass, 0 fail → $H = 0.0$ (pure group)

NO (S2,S4,S5,S6,S8,S10): 1 pass, 5 fail → $H = 0.65$

$$IG = 1.0 - [(4/10) \times 0.0 + (6/10) \times 0.65]$$

Weighted average entropy of the two child groups.

$$IG = 1.0 - 0.39 = 0.61 \rightarrow \text{This split is very useful}$$



Which Question Wins?

The algorithm ran that same calculation for all 5 possible questions. The winner becomes the root node.

Question	YES group	NO group	Info Gain
study_hours > 1	5p 2f H=0.86	0p 3f H=0.0	0.3958
study_hours > 2	5p 0f H=0.0	0p 5f H=0.0	1.0
study_hours > 3	4p 0f H=0.0	1p 5f H=0.65	0.61
study_hours > 4	2p 0f H=0.0	3p 5f H=0.95	0.2365
attendance==High	3p 2f H=0.97	2p 3f H=0.97	0.029

P= Pass
F= Fail

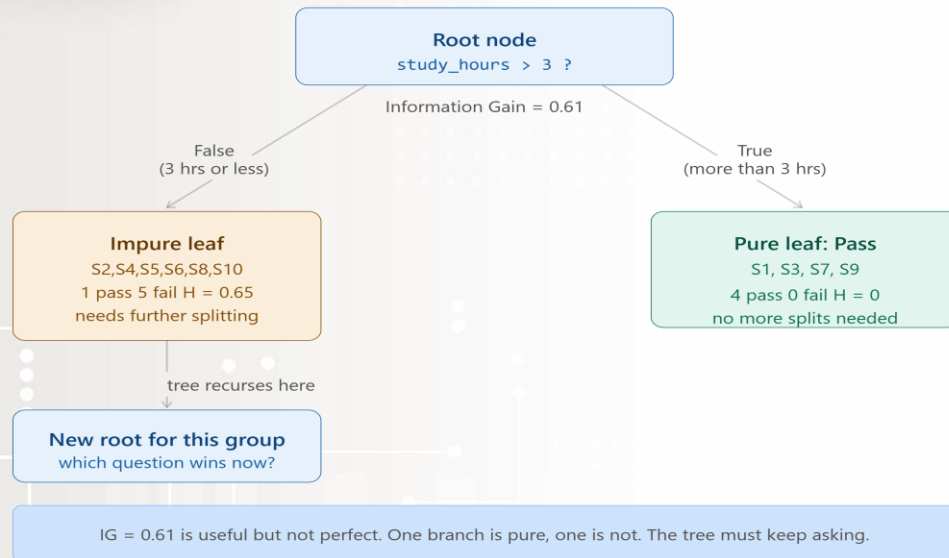
Winner: study_hours > 2 IG = 1.0 Both groups completely pure. Every student who studied more than 2 hours passed. Every student who studied 2 hours or less failed. This becomes the root node.



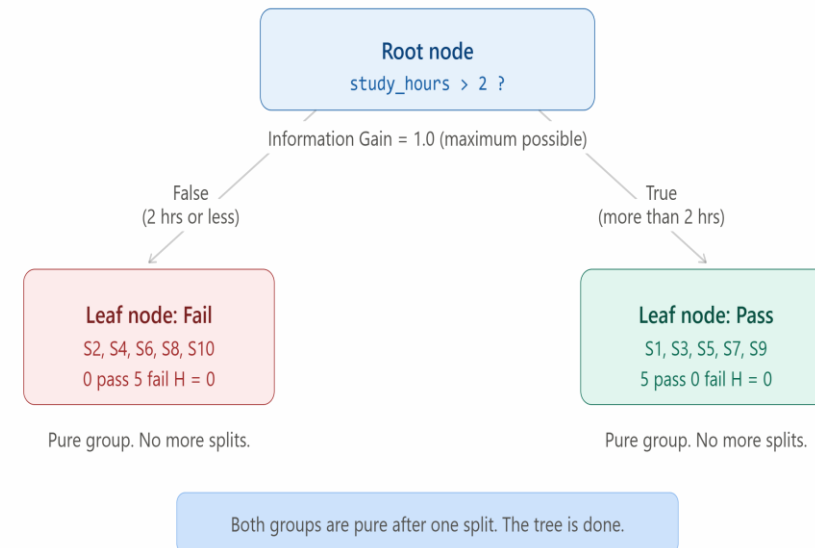
What Each Split Actually Produces

Same dataset. Same algorithm. Different question. The result looks completely different.

study_hours > 3 IG = 0.61



study_hours > 2 IG = 1.0 (winner)



IG = 0.61: one pure branch, one impure branch, tree must recurse. **IG = 1.0:** both branches pure, tree is done in one split.

Now Let's See It in Python

```
import pandas as pd
from sklearn.tree import DecisionTreeClassifier, export_text

# Load the student performance data
df = pd.read_csv('student_performance.csv')

# Split features from target
X = df[['study_hours', 'attendance', 'prev_grade']]
y = df['result'] # pass or fail

# Train the Decision Tree
model = DecisionTreeClassifier(max_depth=3, criterion='entropy')
model.fit(X, y)

# Predict a new student
new_student = [[4, 'High', 65]] # 4hrs, high attendance, prev 65%
print(model.predict(new_student)) # --> ['pass']
```

One Tree Is Just the Beginning



(B) Random Forest
Multiple Trees built
in parallel using
random
data/features subset
Final outcome by
majority

Decision Tree

Breiman et al. (1984) CART; Quinlan (1986) ID3

- Splits chosen by maximising information gain (ID3/C4.5) or minimising Gini impurity (CART).
- Single deterministic model, same data, same tree.
- High variance: small data changes can produce a completely different tree.

The instability problem → solved by combining many trees

Random Forest

Breiman (2001). Random Forests. Machine Learning, 45(1), 5–32.

- Each tree trains on a bootstrap sample (random rows, drawn with replacement) + random subset of features.
- Still uses Gini or information gain per split.
- Final prediction = majority vote (classification) or mean (regression).
- Variance drops because averaging uncorrelated trees cancels individual errors.

One Problem: Overfitting

The overfitting problem

Imagine studying a past paper so hard you memorise every answer word for word. Then the lecturer changes the wording slightly. You fail because you memorised, not understood.

An overfit tree does the same thing. It memorised the training students. It can't generalise to new ones.

The fix: stopping criteria

We give the tree rules that stop it growing too deep.

Maximum depth: the tree can only ask a set number of questions.

Minimum samples: a branch can only split if it has enough students in it.

In sklearn: `max_depth=3, min_samples_split=5`

References

Mitchell, T.M. (1997). Machine Learning. McGraw-Hill. ISBN: 978-0070428072

Quinlan, J.R. (1986). Induction of Decision Trees. Machine Learning, 1(1), 81–106. DOI: 10.1007/BF00116251

Shannon, C.E. (1948). A Mathematical Theory of Communication. Bell System Technical Journal, 27(3), 379–423.

Goodfellow, I., Bengio, Y., Courville, A. (2016). Deep Learning. MIT Press. ISBN: 978-0262035613

Sutton, R.S. & Barto, A.G. (2018). Reinforcement Learning: An Introduction (2nd ed.). MIT Press. ISBN: 978-0262039246

Breiman, L., Friedman, J., Olshen, R., Stone, C. (1984). Classification and Regression Trees (CART). Wadsworth. ISBN: 978-0412048418

Hastie, T., Tibshirani, R., Friedman, J. (2009). The Elements of Statistical Learning (2nd ed.). Springer. DOI: 10.1007/978-0-387-84858-7

AN INITIATIVE OF



science, technology
& innovation

Department:
Science, Technology and Innovation
REPUBLIC OF SOUTH AFRICA

MANAGED BY



THANK YOU

NICIS

NATIONAL INTEGRATED
CYBERINFRASTRUCTURE SYSTEM

DIRISA